

June 18th, 2025

**Integrative semester B, 2025
(10143)**

SoldiCare

**Ambient Invisible
Intelligence**

Lecturers:

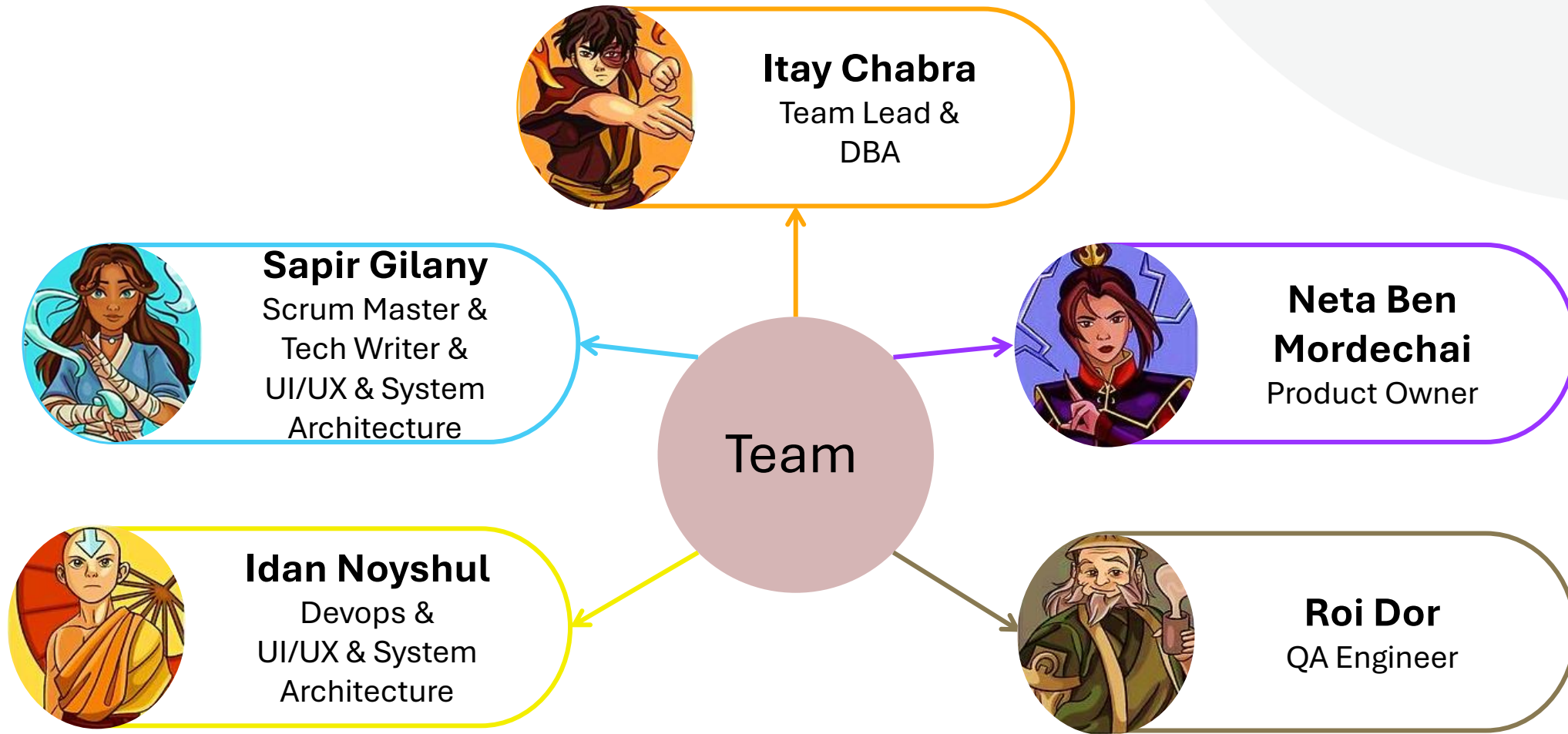
Deganit Armon, Tom Cohen

[System Video](#)

[Figma](#)



Meet the team



System Overview



Problem Statement & General System Goal

Problem

Lack of real-time medical monitoring in combat zones.

Challenge

Soldiers are not always aware of the deterioration in their health condition.

Consequences

Reduced operational readiness and increased risk to human life.

Objective

Ensure the health and combat readiness of soldiers in the field.

Solution

Advanced monitoring with an intuitive smart interface and instant alerts.

Scope of System

Core Functionality

Real-time health monitoring
and alert system

Security and Data Protection

No guaranteed encryption or
advanced access control

Operational Constraints

Requires stable connectivity,
supplements medical
evaluations

Limitations

No medical diagnoses, data
transmission may be affected
by external factors

Understanding the Users

Soldiers

Monitoring their
physical and
health status



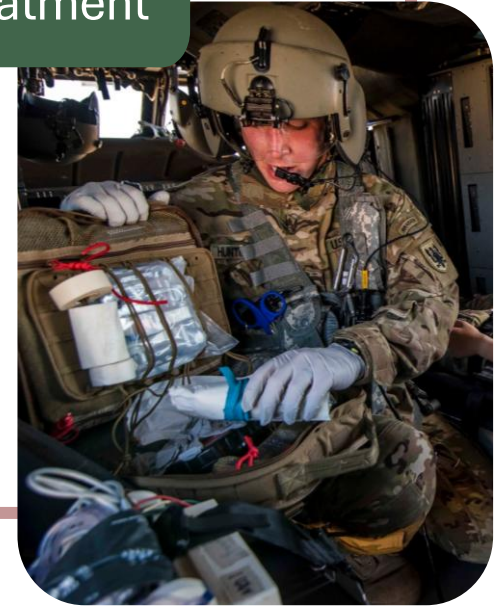
Commanders

Field force
management
and unit
readiness



Military doctors

Prioritization of
field treatment



Research & Conceptual Planning

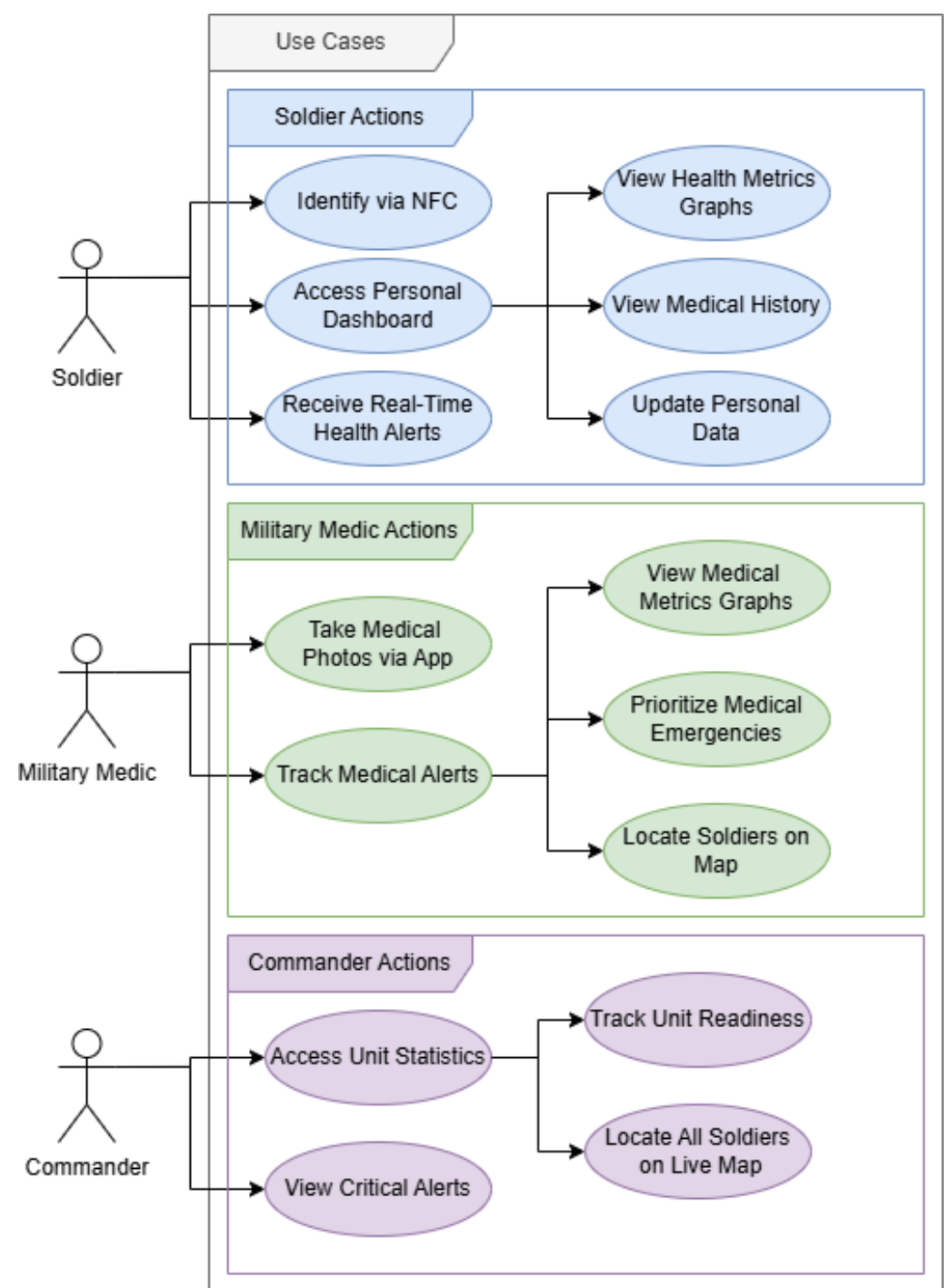


Market Research & Industry Insights

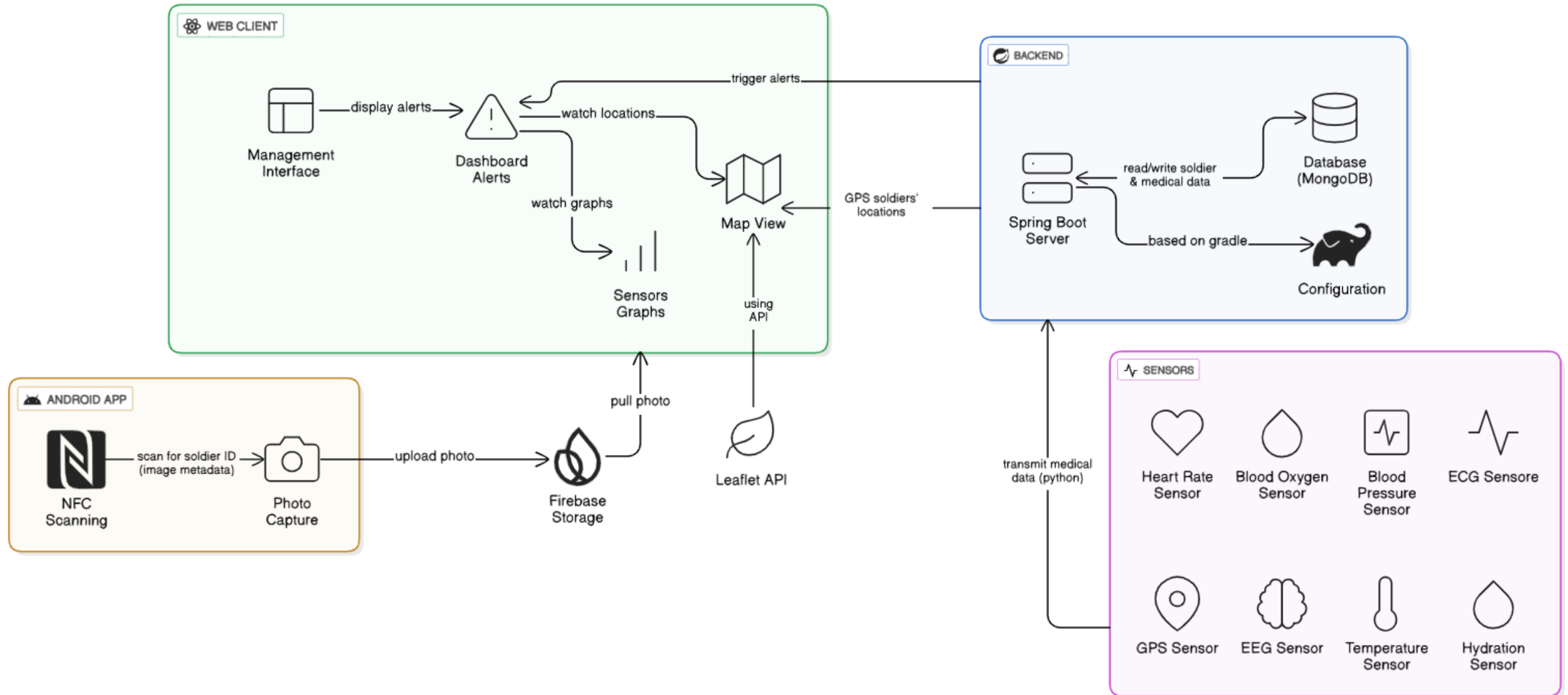
CriterionSystem	IoMT	HRS	Equivital
Type of medical monitoring	Pulse, blood pressure, oxygen saturation, stress	Blood pressure, oxygen saturation, cardiac metrics	Pulse, respiration, skin temperature, movement
Suitability for military use	✓	✗	✓
Data analysis capability	AI	Basic medical	Advanced physiological
Integration with command systems	✓	✗	✓
User interface	Adapted to military terminology	Intuitive for medical treatment	Adapted to field conditions



Use Case Analysis & System Workflow



Core Functionality & System Architecture



Non-Functional Requirements

Usability

- ✓ Visual Data Representation
- ✓ Responsive System Design

Reliability

- ✓ Medical-Grade Alert Classification

Performance

- ✓ Concurrent Session Handling

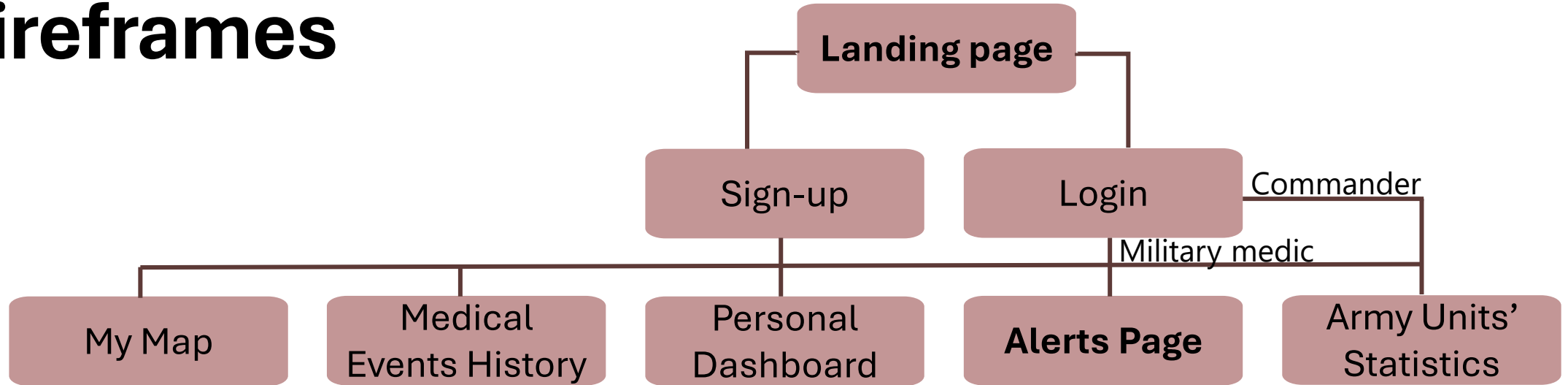
Supportability

- ✓ Extensibility & Scalability
- ✓ Logging & Debugging Mechanisms

Frontend Development & System Interfaces



User Interface Design & Conceptual Wireframes

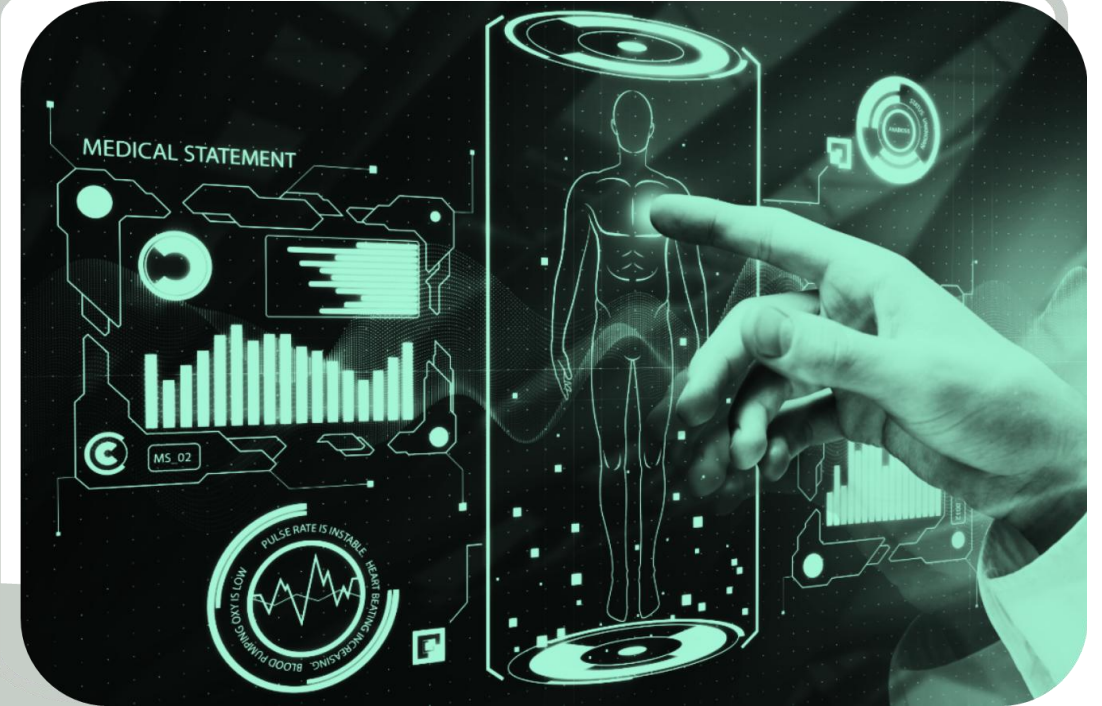


Design Goal

- ✓ Critical medical data monitoring
- ✓ No cumbersome navigation – localized and centralized data
- ✓ Real-time decision support
- ✓ Separation between historical data and current information

Client-Side Functionalities

- ✓ Real-time medical monitoring for soldiers
- ✓ Receives vitals from biometric sensors
- ✓ Smart alerts, graphs, and live map
- ✓ View history and generate unit stats



Client-Side Code Highlights

Custom Hook

Efficiently manages soldier health data for quick access to medical history and alerts

- ✓ Manages soldier and unit state
- ✓ Auto-fetch on data change
- ✓ Parallel fetching using Promise.all
- ✓ Returns structured data object

```
export const useSoldierData = (  
  soldiers: Soldier[],  
  selectedId: string | null,  
  systemID: string,  
  email: string  
) => {  
  const [activeSoldier, setActiveSoldier] = useState<Soldier | null>(null);  
  const [soldierUnit, setSoldierUnit] = useState<UnitData | null>(null);  
  const [snapshotHistory, setSnapshotHistory] = useState<Snapshot[]>([]);  
  const [loading, setLoading] = useState<boolean>(false);  
  
  useEffect(() => {  
    const fetchSoldierData = async () => {  
      setLoading(true);  
  
      const soldier = Idan Noyshul, yesterday • custom Hook  
        soldiers.find((s) => s.id.objectId === selectedId) ??  
        soldiers[0] ??  
        null;  
  
      setActiveSoldier(soldier);  
  
      if (soldier) {  
        try {  
          const [unitRaw, snapshots] = await Promise.all([  
            getSoldierUnit(  
              soldier.id.systemID,  
              soldier.id.objectId,  
              systemID,  
              email  
            ),  
            getSoldierSnapshot(  
              soldier.id.systemID,  
              soldier.id.objectId,  
              systemID,  
              email  
            ),  
          ];  
        }  
      }  
    }  
  });  
}
```

Client-Side Code Highlights

Map View

Provides a real-time unit overview with health statuses and hotspot markers.

- ✓ Fetches unit data from API
- ✓ Renders polygons and hotspot markers
- ✓ Handles map clicks (shows coordinates)
- ✓ Integrates unit stats modal

```
33 export default function MapView() {
34   const { systemID, email } = useAppContext();
35   const [clickedPos, setClickedPos] = useState<LatLngTuple | null>(null);
36   const [open, setOpen] = React.useState<boolean>(true);
37   const [allUnits, setAllUnits] = useState<UnitData[]>([]);
38   const [unitsToDisplay, setUnitsToDisplay] = useState<UnitData[]>([]);
39   const [isLoading, setLoading] = useState<boolean>(false);
40   const handleOpen = () => setOpen(true);
41   const handleClose = () => setOpen(false);
42
43   const fetchUnits = async () => {
44     try {
45       setLoading(true);
46       const rawUnits = await getArmyUnits(systemID, email);
47
48       const formattedUnits: UnitData[] = rawUnits.map((unit: any) => ({
49         id: unit.id,
50         name: unit.objectDetails.name,
51         color: unit.objectDetails.color,
52         polygon: unit.objectDetails.polygon,
53         hotspots: [],
54         status: unit.status,
55         severelyWounded: unit.objectDetails.severelyWounded,
56         lightlyWounded: unit.objectDetails.lightlyWounded,
57         immobileUnconscious: unit.objectDetails.immobileUnconscious,
58         dead: unit.objectDetails.dead,
59         combatReadyTroops: unit.objectDetails.combatReadyTroops,
60       }));
61
62       setAllUnits(formattedUnits);
63       setUnitsToDisplay(formattedUnits);
64     } catch (err) {
65       console.error("Error fetching units:", err);
66     } finally {
67       setLoading(false);
68     }
69   };
70 }
```

Field Medical Photo Capture App

- ✓ NFC scans to link photos to soldiers
- ✓ Real-time injury documentation by field medics
- ✓ Image storage in Firebase cloud
- ✓ Instant medical team access to uploaded photos



App Code Highlights

Metadata handling

```
StorageMetadata metadata = new StorageMetadata.Builder()  
    .setContentType("image/jpeg")  
    .setCustomMetadata("SoldierID", scannedNfcId)  
    .setCustomMetadata("SystemID", "2025b.Itay.Chabra")  
    .setCustomMetadata("captureDate", new java.text.SimpleDateFormat(  
        pattern: "yyyy-MM-dd", java.util.Locale.getDefault()).format(new java.util.Date()))  
    .setCustomMetadata("captureTime", new java.text.SimpleDateFormat(  
        pattern: "HH:mm:ss", java.util.Locale.getDefault()).format(new java.util.Date()))  
    .setCustomMetadata("captureDateTime", new java.text.SimpleDateFormat(  
        pattern: "yyyy-MM-dd HH:mm:ss", java.util.Locale.getDefault()).format(new java.util.Date()))  
    .setCustomMetadata("UserID", "1234567")  
    .setCustomMetadata("deviceModel", android.os.Build.MODEL)  
    .build();
```



App Code Highlights

Compression with Orientation Correction

Image compression that preserves correct orientation based on metadata.

- ✓ EXIF metadata preservation and manipulation
- ✓ Memory-efficient bitmap processing
- ✓ Configurable compression with quality control

```
private File compressImage(File originalFile) {
    try {
        // Get the original image orientation
        ExifInterface exif = new ExifInterface(originalFile.getAbsolutePath());
        int orientation = exif.getAttributeInt(ExifInterface.TAG_ORIENTATION, ExifInterface.ORIENTATION_NORMAL);

        Log.d(TAG, msg: "Original image orientation: " + orientation);

        // Load the image
        Bitmap bitmap = BitmapFactory.decodeFile(originalFile.getAbsolutePath());

        if (bitmap == null) {...}

        // Rotate the bitmap if needed
        Bitmap rotatedBitmap = rotateBitmapIfNeeded(bitmap, orientation);

        // If rotation created a new bitmap, recycle the original
        if (rotatedBitmap != bitmap) {...}

        // Create compressed file
        File compressedDir = new File(getExternalFilesDir(Environment.DIRECTORY_PICTURES), child: "compressed");
        if (!compressedDir.exists() && !compressedDir.mkdirs()) {...}

        File compressedFile = new File(compressedDir, child: "compressed_" + originalFile.getName());

        // Compress and save
        FileOutputStream fos = new FileOutputStream(compressedFile);

        // Compress with 60% quality (adjust as needed: 0-100)
        boolean success = rotatedBitmap.compress(Bitmap.CompressFormat.JPEG, COMPRESSION_QUALITY, fos);

        fos.flush();
        fos.close();
        rotatedBitmap.recycle(); // Free memory

        if (!success) {...}

        // Set the correct orientation in the compressed file's EXIF data
        ExifInterface compressedExif = new ExifInterface(compressedFile.getAbsolutePath());
        compressedExif.setAttribute(ExifInterface.TAG_ORIENTATION, String.valueOf(ExifInterface.ORIENTATION_NORMAL));
        compressedExif.saveAttributes();

        Log.d(TAG, msg: "Image compressed successfully with correct orientation");
        return compressedFile;
    } catch (Exception e) {...}
}
```

NFC Code Highlight

NDEF tag reading

Reads and parses NDEF tags to retrieve encoded soldier identification data.

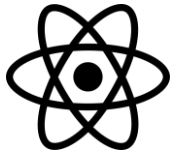
- ✓ Proper NDEF record parsing with language code handling
- ✓ Understanding of NFC data format specifications
- ✓ Robust error handling without crashing



```
private String readNfcTag(Tag tag) {  
    Log.d(TAG, msg: "Reading NFC tag...");  
  
    try {  
        // First try to read NDEF data  
        Ndef ndef = Ndef.get(tag);  
        if (ndef != null) {  
            Log.d(TAG, msg: "NDEF tag detected");  
            ndef.connect();  
  
            if (ndef.isConnected()) {  
                NdefMessage ndefMessage = ndef.getNdefMessage();  
                if (ndefMessage != null) {  
                    NdefRecord[] records = ndefMessage.getRecords();  
                    Log.d(TAG, msg: "Found " + records.length + " NDEF records");  
  
                    for (NdefRecord record : records) {  
                        Log.d(TAG, msg: "Processing NDEF record, TNF: " + record.getTnf());  
  
                        if (record.getTnf() == NdefRecord.TNF_WELL_KNOWN &&  
                            java.util.Arrays.equals(record.getType(), NdefRecord.RTD_TEXT)) {...}  
                        // Also try reading as UTF-8 string directly for some tag types  
                        else if (record.getTnf() == NdefRecord.TNF_WELL_KNOWN) {  
                            try {  
                                String directText = new String(record.getPayload(), StandardCharsets.UTF_8);  
                                if (directText.length() > 0 && !directText.contains("\u0000")) {  
                                    Log.d(TAG, msg: "Read as direct UTF-8: " + directText);  
                                    ndef.close();  
                                    return directText;  
                                }  
                            } catch (Exception e) {  
                                Log.d(TAG, msg: "Could not read as direct UTF-8");  
                            }  
                        }  
                    }  
                }  
                ndef.close();  
                Log.d(TAG, msg: "No readable text records found in NDEF message");  
            } else {  
                Log.d(TAG, msg: "Not an NDEF tag, trying to read tag ID");  
            }  
  
            // Fallback: return tag ID as hex string  
            byte[] tagId = tag.getId();  
            if (tagId != null && tagId.length > 0) {  
                String hexId = bytesToHex(tagId);  
                Log.d(TAG, msg: "Using tag ID as fallback: " + hexId);  
                return hexId;  
            }  
        } catch (Exception e) {...}  
  
        Log.w(TAG, msg: "Could not read any data from NFC tag");  
        return null;  
    }  
}
```

Technologies Behind the Frontend

Frontend Web



React +
TypeScript



Material
UI



React
Hooks



Leaflet
API



Recharts
Library

Image Storage & Metadata Handling



Firebase
Storage



ExifInterface



Bitmap &
Graphics
APIs

NFC Technology



Android
NFC API



NDEF (NFC
Data Format)



Foreground
Dispatch
System

Android Application



Kotlin



CameraX



FileProvider



Material
Design
Components



Constraint
Layout

Backend Development & System Operations



Python-Based Sensor Data Simulation

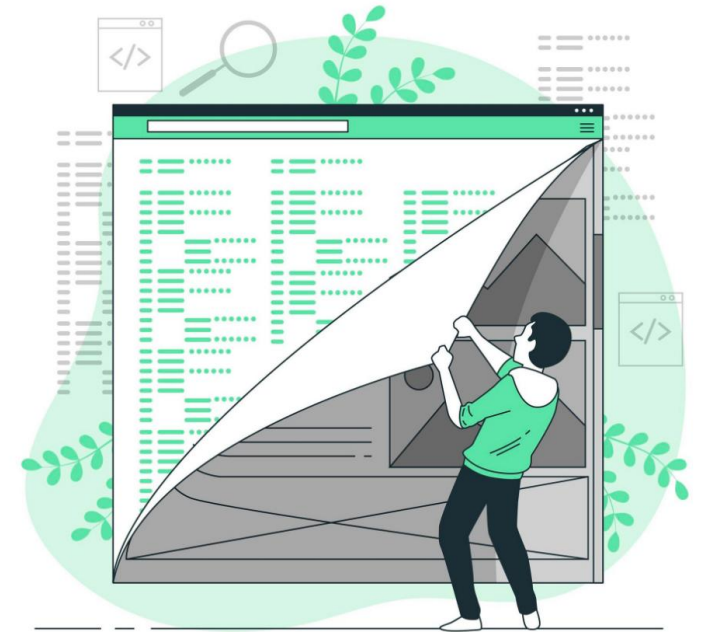
(for development and POC purposes)

- ✓ Simulates biometric sensor data with randomized values
- ✓ Automatic soldier identification & unit linking
- ✓ Send critical health alerts via command
- ✓ Utilizes Python modules: requests, JSON, logging
- ✓ Runs continuously for real-time simulation testing



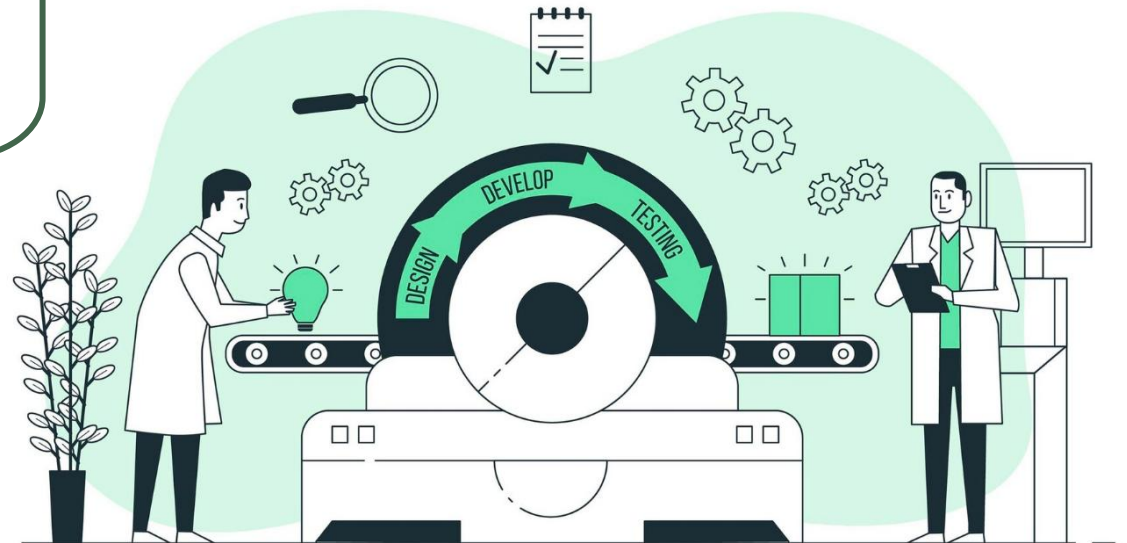
Custom Backend Enhancements for Our System

- ✓ AlertController – RESTful endpoint for managing alerts, enabling retrieval based on user and pagination settings.
- ✓ AlertService – Handles alert retrieval, interacting with CommandService to filter critical commands.
- ✓ Snapshot History – Maintains historical alert records for context and accessibility.
- ✓ Related Soldiers Retrieval – Extracts data on associated soldiers to provide a comprehensive situational overview.

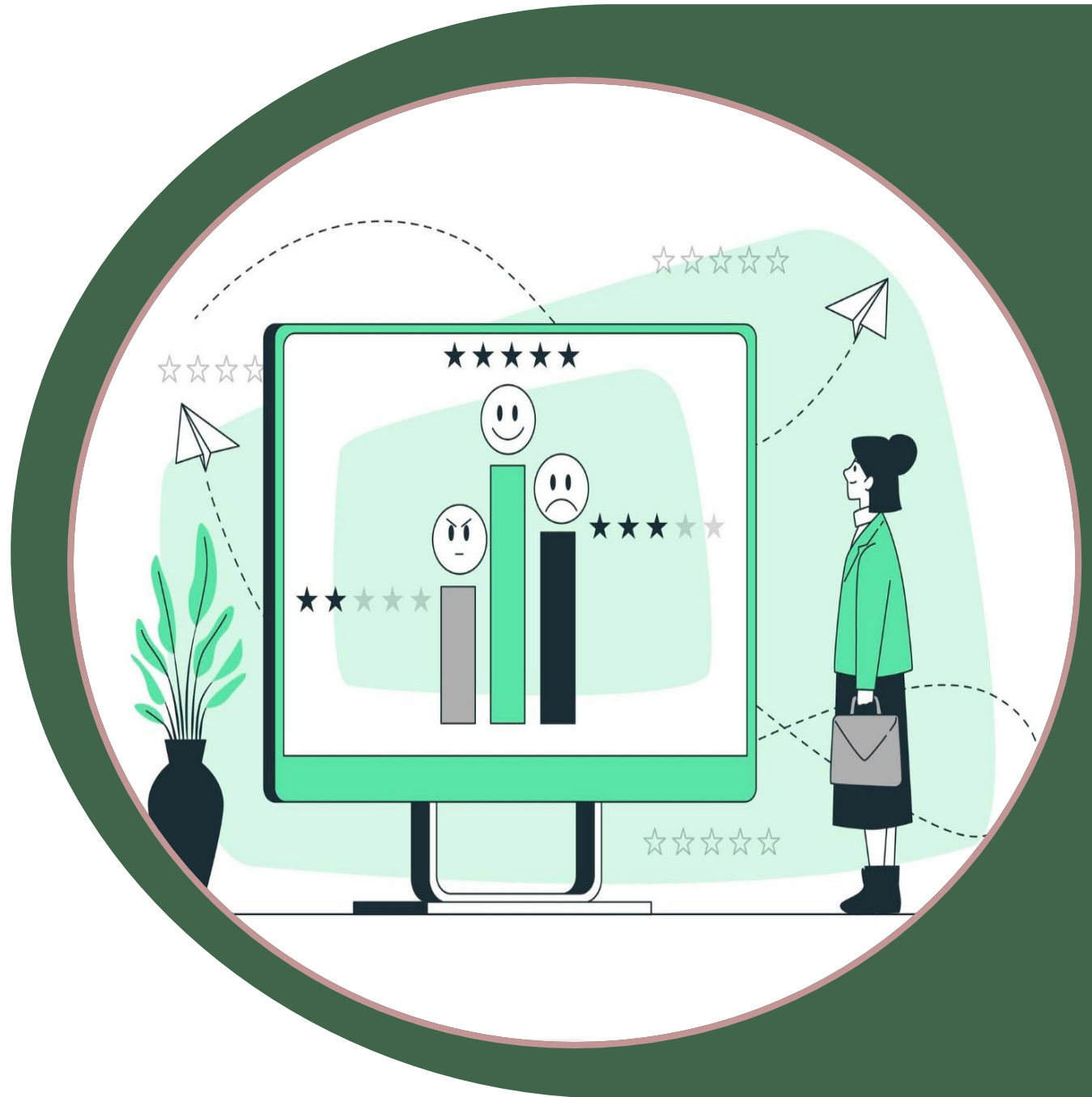


Unit Tests & Quality Assurance

- ✓ Unit & Integration Testing for System Components
- ✓ Mock-based Testing Using Mockito
- ✓ Spring Boot & REST API Validation
- ✓ Structured Gherkin Test Writing (BDD – Behavior-Driven Development)
- ✓ Automated Test Execution & Continuous Validation



Final Review



Kanban Board #1

03/06/2025



SoldiCare ☆ Workspace visible Board

Backlog

- devops DB product owner
- Code Review QA
- reports, documents, powerpoint
- architecture Development
- UI/UX
- All the Labels
- Functionalities tailored to the unique needs of our system
- python requests sending jsons
- + Add a card

TO-DO

- Development
sensor simulators - python script
Jun 7
- Development
Soldier Initializer Modify
Jun 7
- Development
army unit initializer modifier
Jun 8
- QA
Modify tests
- QA Development
Performance Tests
- Development
getAlerts Route
Jun 11
- reports, documents, powerpoint
Build Readme
0/3
- reports, documents, powerpoint
powerpoint end of project including everything on our system
- Development
Build text file for client simulation
- + Add a card

In Progress

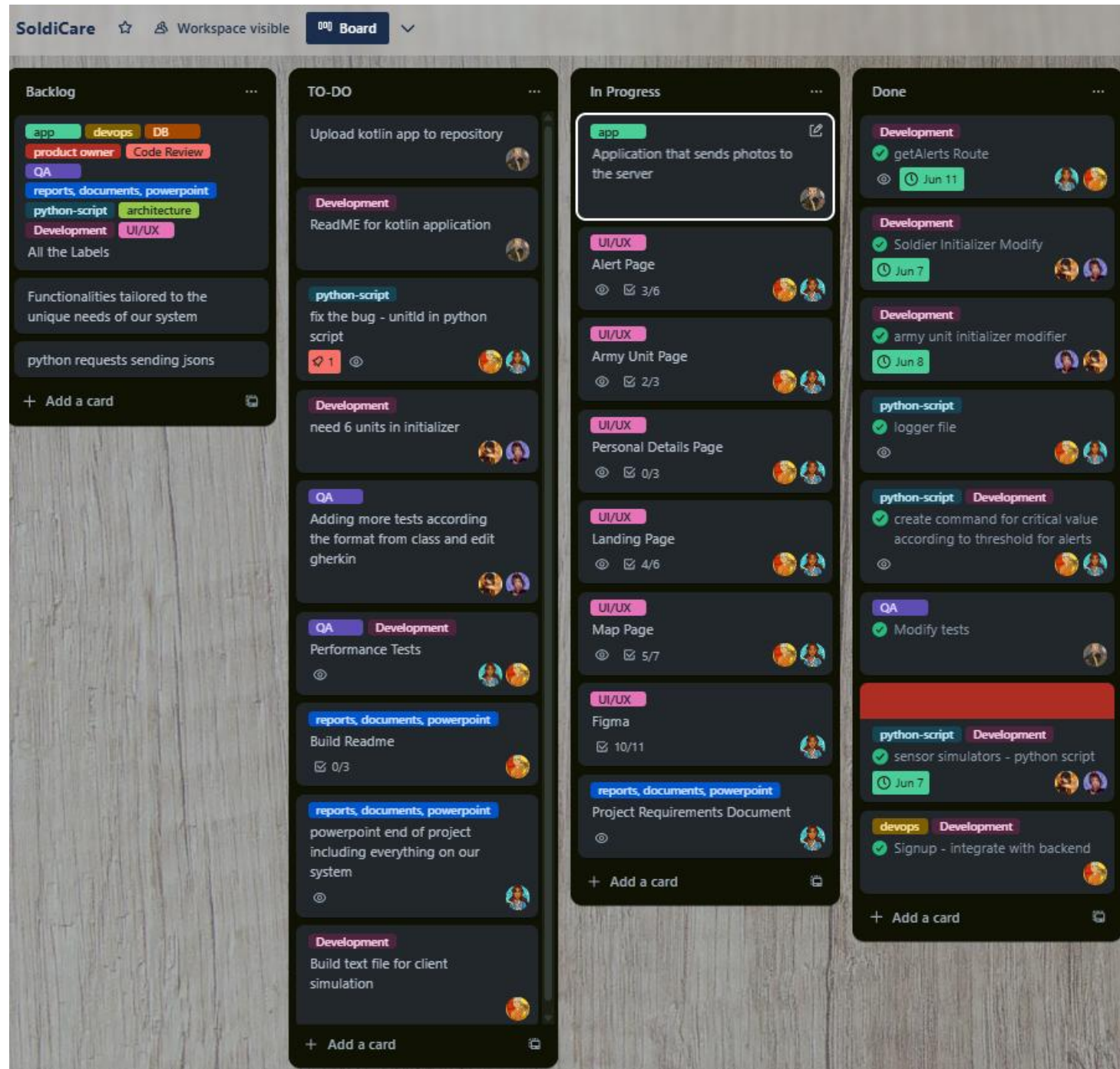
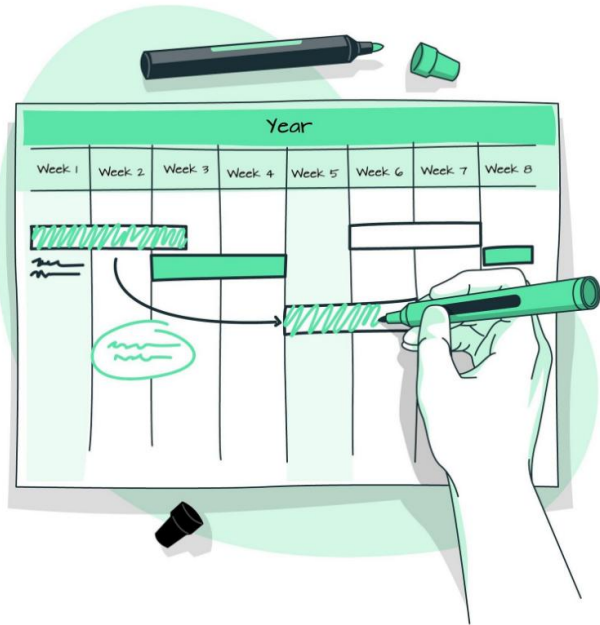
- devops Development
Signup - integrate with backend
- UI/UX
Alert Page
3/6
- UI/UX
Army Unit Page
2/3
- UI/UX
Personal Details Page
0/3
- UI/UX
Landing Page
0/3
- UI/UX
Map Page
5/7
- UI/UX
Figma
9/10
- reports, documents, powerpoint
Project Requirements Document
- + Add a card

Done

- + Add a card

Kanban Board #2

10/06/2025



Kanban Board #3

14/06/2025



SoldiCare Workspace visible Board

Backlog

- app devops DB
- product owner Code Review
- QA
- reports, documents, powerpoint
- python-script architecture
- Development UI/UX
- All the Labels
- Functionalities tailored to the unique needs of our system
- python requests sending jsons
- Artemis
- + Add a card

TO-DO

- Development
Soldier_INITIALIZER - Watch Description
- Development
need 6 units in initializer
- Going over the improvements from sprint 3
- Modify users initializers - intend Joana@soldicare.com do operator@soldicare.com same to other roles
- QA Development
Performance Tests
- reports, documents, powerpoint
General Summarize Sheet - read description - write in plain word in english
- reports, documents, powerpoint
Technology Sheet (app, test) - write in plain word in english
- reports, documents, powerpoint
powerpoint slide - technologies app, test client
- python-script
fix the bug - unitid in python script

TO-DO

- reports, documents, powerpoint
Technology Sheet (app, test) - write in plain word in english
- reports, documents, powerpoint
powerpoint slide - technologies app, test client
- python-script
fix the bug - unitid in python script
- reports, documents, powerpoint
Build Readme
- reports, documents, powerpoint
Technology Sheet (web) - write in plain word in english
- Development
Build text file for client simulation
- reports, documents, powerpoint
powerpoint slide - technologies web client
- reports, documents, powerpoint
powerpoint end of project including everything on our system

In Progress

- QA
Adding more tests according the format from class and edit gherkin
- UI/UX
Alert Page
- UI/UX
Army Unit Page
- UI/UX
Personal Details Page
- UI/UX
Landing Page
- UI/UX
Map Page
- UI/UX
Figma
- reports, documents, powerpoint
Project Requirements Document

Done

- Development
getAlerts Route
- Upload Android app to repository
- app
Application that sends photos to the server
- Development
ReadME for kotlin application
- Development
Soldier_INITIALIZER Modify
- Development
army unit initializer modifier
- python-script
logger file
- python-script Development
create command for critical value according to threshold for alerts
- QA
Modify tests

Kanban Board #4

17/06/2025



SoldiCare ☆ Workspace visible

Backlog

- app devops DB
- product owner Code Review
- QA
- reports, documents, powerpoint
- python-script architecture
- Development UI/UX
- All the Labels
- Artemis
- MedicalHistory page
- + Add a card

Done

- reports, documents, powerpoint
 - ✓ Project Requirements Document
- reports, documents, powerpoint
 - ✓ powerpoint end of project including everything on our system
- reports, documents, powerpoint
 - ✓ General Summarize Sheet - read description - write in plain word in english
- UI/UX
 - ✓ Map Page
- UI/UX
 - ✓ Landing Page
- UI/UX
 - ✓ Army Unit Page
- UI/UX
 - ✓ Personal Details Page
- UI/UX
 - ✓ Alert Page
- ✓ Going over the improvements from sprint 3

Done

- reports, documents, powerpoint
 - ✓ Build Readme
- Development
 - ✓ Build text file for client simulation
- Development
 - ✓ fetch image from firebase to integrate with client
- reports, documents, powerpoint
 - ✓ Technology Sheet (web) - write in plain word in english
- reports, documents, powerpoint
 - ✓ Technology Sheet (app, test) - write in plain word in english
- reports, documents, powerpoint
 - ✓ powerpoint slide - technologies app, test client
- upload 18 images according to serial in the initializer
- UI/UX
 - ✓ Figma
- reports, documents, powerpoint
 - ✓ powerpoint slide - technologies web client

Done

- reports, documents, powerpoint
 - ✓ powerpoint slide - technologies web client
- Development
 - ✓ creating a route for find soldier by serial
- Development
 - ✓ Soldier Initializer
- Development
 - ✓ need 6 units in initializer
- python-script
 - ✓ fix the bug - unitId in python script
- QA
 - Adding more tests according the format from class and edit gherkin
- Development
 - ✓ getAlerts Route
- Upload Android app to repository
- app
 - Application that sends photos to the server

Done

- ReadME for kotlin application
- Development
 - ✓ Soldier Initializer Modify
- Development
 - ✓ army unit initializer modifier
- python-script
 - ✓ logger file
- python-script Development
 - ✓ create command for critical value according to threshold for alerts
- QA
 - ✓ Modify tests
- python-script Development
 - ✓ sensor simulators - python script
- devops Development
 - ✓ Signup - integrate with backend
- QA
 - Gherkin for unit tests

Project Retrospective: Achievements & Improvements

Achievements

- ✓ Effective Kickoff Meetings
- ✓ Utilizing multiple collaboration platforms
- ✓ Structured Git workflow
- ✓ Dividing tasks into small sub-teams
- ✓ Commitment to deadlines and quality
- ✓ Thorough code review processes



Improvements

- ✓ Strengthening team communication and synchronization
- ✓ Encouraging proactive engagement and responsibility
- ✓ Enhancing feedback culture
- ✓ Improving task execution within timelines
- ✓ Fostering innovation and new initiatives



Thanks for your attention

